



Guía n°

Configurando CORS en el API Gateway

ESCUELA DE INFORMÁTICA Y TELECOMUNICACIONES

Guía n°

Configurando CORS en el API Gateway

Sigla	Asignatura	Experiencia de Aprendizaje
Tiempo	Modalidad de Trabajo	Indicadores de Logro

Contexto

¿Qué es CORS y para qué sirve?

CORS (Cross-Origin Resource Sharing) es un mecanismo de seguridad basado en cabeceras HTTP que permite a un servidor indicar desde qué orígenes (dominios, esquemas o puertos) se permite la carga de sus recursos.

El problema: Same-Origin Policy (SOP)

Por defecto, los navegadores aplican la Política del Mismo Origen. Esto significa que una aplicación web alojada en <https://mi-app.com> no puede hacer una petición (por ejemplo, un fetch o XMLHttpRequest) a <https://api.mi-aws.com> porque los dominios son diferentes. El navegador bloquea la respuesta por seguridad para prevenir ataques como CSRF.

La solución: CORS

CORS "relaja" esta restricción de forma controlada. Cuando tu frontend hace una petición a tu API en AWS:

1. Petición Simple (Simple Request): El navegador envía la petición con la cabecera Origin. Si el servidor responde con Access-Control-Allow-Origin coincidente, el navegador entrega la respuesta al usuario.
2. Petición con Preflight (Preflight Request): Para peticiones complejas (ej. PUT, DELETE, o headers personalizados como Authorization o Content-Type: application/json), el navegador envía primero una petición automática de tipo



OPTIONS. El servidor debe aprobarla explícitamente antes de que el navegador envíe la petición real.

Diferencia Crítica: REST APIs vs. HTTP APIs

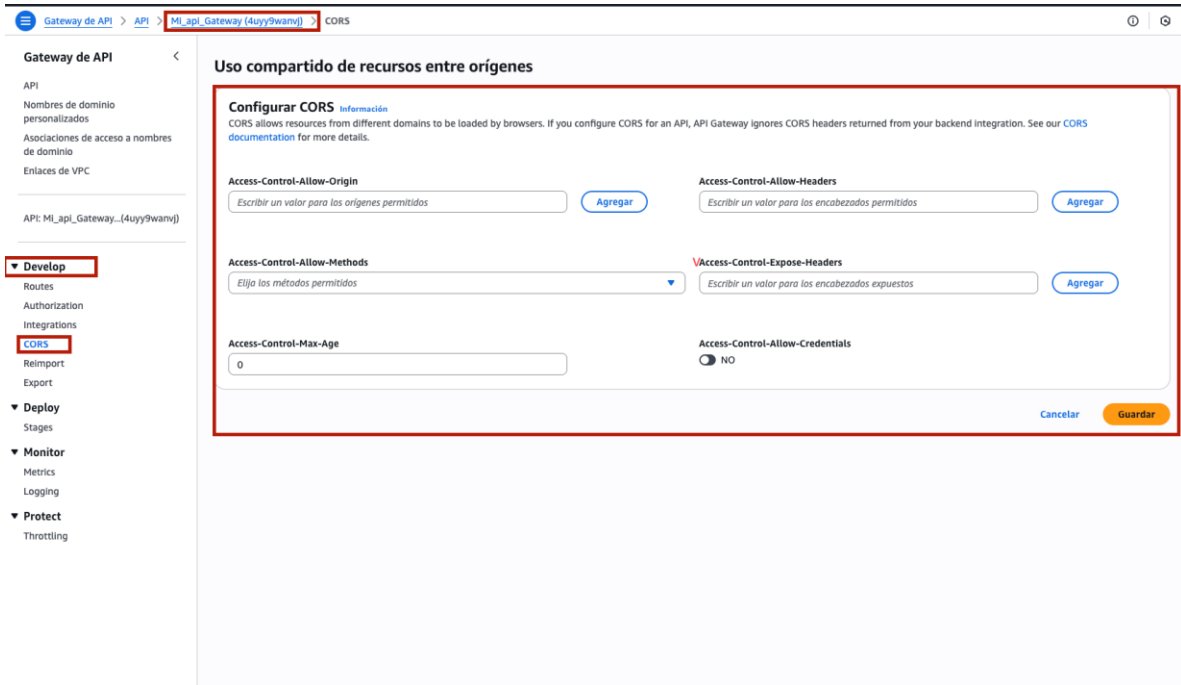
En AWS API Gateway existen dos tipos de APIs y la configuración de CORS es radicalmente distinta:

- HTTP APIs (v2): Son más nuevas, baratas y sencillas. AWS maneja el CORS casi automáticamente.
- REST APIs (v1): Son las "clásicas", más completas. El manejo de CORS es manual y a menudo requiere configuración tanto en el Gateway como en tu código backend (Lambda).

Comenzamos

Pasos:

1. Ve a la consola de API Gateway y selecciona tu HTTP API.
2. En el menú lateral izquierdo, bajo la sección Develop (Desarrollo), haz clic en CORS.
3. Haz clic en Configure (Configurar).
4. Rellena los campos clave:



Access-Control-Allow-Origin: Especifica los dominios permitidos.

- *Desarrollo:* * (Permite todo).
- *Producción:* <https://tu-dominio.com> (Especifica la URL exacta de tu frontend).

Access-Control-Allow-Headers: Lista de headers que tu frontend enviará.

- Recomendados: Content-Type, Authorization, X-Amz-Date, X-API-Key.

Access-Control-Allow-Methods: Los verbos HTTP que usas (ej. GET, POST, PUT, DELETE, OPTIONS).

Configuración en REST APIs (v1)

Aquí es donde la mayoría de los arquitectos encuentran problemas. La configuración depende de si usas Integración Proxy o Integración No-Proxy.

Habilitar el método OPTIONS (El Preflight):



Ve a tu API -> Resources.

Selecciona el recurso (ej. /datos).

Haz clic en Enable CORS en el menú de acciones o botón directo.

Marca las casillas "Access-Control-Allow-Methods" y pon tu origen en "Access-Control-Allow-Origin" (ej. '*').

Esto crea un método OPTIONS en tu recurso que responde automáticamente el preflight con código 200.

Links de interés

1. Definición y Conceptos

- ¿Qué es CORS? (Explicación general):
<https://aws.amazon.com/what-is/cross-origin-resource-sharing/>

2. Para REST APIs (v1)

- Guía principal de activación (Habilitar CORS en un recurso):
<https://docs.aws.amazon.com/apigateway/latest/developerguide/how-to-cors.html>



- Cómo habilitarlo desde la Consola de AWS (Paso a paso visual):
<https://docs.aws.amazon.com/apigateway/latest/developerguide/how-to-cors-console.html>

3. Para HTTP APIs (v2)

- Configuración de CORS para API HTTP:
<https://docs.aws.amazon.com/apigateway/latest/developerguide/http-api-cors.html>